CPU simulada de 8 bits

Guia para entender e apresentar o trabalho

Jander Luis Magalhães Coutinho
Arthur Fernandes Viana

UFC Sobral | Arquitetura de Computadores | 2026.2

1. O que a CPU faz

A CPU simulada executa uma lista de instruções. Essas instruções carregam valores nos registradores, fazem cálculos, gravam dados na memória, mudam o endereço da próxima instrução ou encerram a execução.

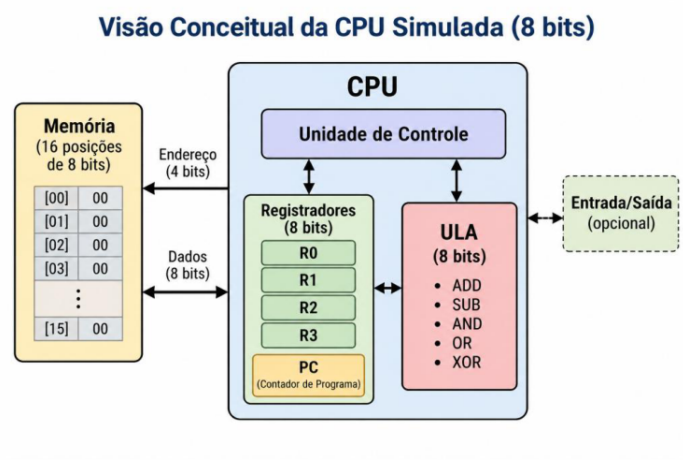


Figura: visão conceitual fornecida no enunciado da disciplina. No código Python, instruções são textos; os valores numéricos são limitados a 8 bits.

Componente	Explicação simples
Registradores R0 a R3	Quatro espaços que guardam os valores usados nas operações.
Memória	16 posições que guardam instruções e dados.
PC	Indica onde está a próxima instrução.
ULA	Faz contas e operações lógicas bit a bit.
Controle	Coordena a busca, a interpretação e a execução.

2. Como uma instrução é executada

Fetch = buscar. A CPU lê a posição da memória indicada pelo PC.
Decode = interpretar. Separa o nome da operação e seus operandos.
Execute = executar. Realiza a operação e atualiza o estado.

Exemplo: ADD R0, R1

Com R0 = 5 e R1 = 8, a CPU busca a instrução ADD e lê os dois registradores. A ULA calcula a soma, e R0 recebe 13. R1 continua com 8, enquanto o PC avança para a próxima instrução.

Instrução	O que acontece
LOAD R0, 5	Coloca o próprio valor 5 em R0.
STORE R0, 15	Copia o valor de R0 para a posição 15 da memória.
ADD / SUB	Soma / subtrai os valores de dois registradores.
AND / OR / XOR	Compara os bits de dois registradores.
JMP 10	A próxima instrução fica na posição 10.
HALT	Encerra; o PC permanece na posição da parada.

3. O que significa ter 8 bits

Cada bit pode ser 0 ou 1. Com oito bits, há 256 combinações, de 00000000 a 11111111. Neste projeto, elas representam números sem sinal entre 0 e 255.

A máscara **valor & 0xFF** mantém os oito bits menos significativos. Por isso, o resultado armazenado de 255 + 1 é 0. Em 0 - 1, fica o padrão 11111111, exibido como 255.

O modelo não possui flags de carry, overflow ou sinal. A lista tem 16 posições, mas as instruções em texto não são codificadas em bytes reais. LOAD usa valor imediato, seguindo o código-base.

4. Programa próprio

O programa começa com 12 e aplica operações aritméticas e lógicas até chegar a 2. Depois, grava esse resultado na posição 15 e o valor 5 na posição 14. Ele usa todas as instruções exigidas.

```
00: LOAD R0, 12
01: LOAD R1, 5
02: ADD R0, R1
03: LOAD R2, 3
04: SUB R0, R2
05: AND R0, R1
06: OR R0, R2
07: XOR R0, R1
08: JMP 10
09: LOAD R0, 99
10: STORE R0, 15
11: STORE R1, 14
12: HALT
```

Das **13 instruções armazenadas, 12 são executadas**. JMP pula o endereço 9, então o valor 99 nunca chega a R0. As posições 14 e 15 ficam livres para os dados.

Operações lógicas

AND produz 1 quando ambos os bits são 1. OR produz 1 quando pelo menos um é 1. XOR produz 1 quando os bits são diferentes.

```
14: 00001110    4: 00000100    7: 00000111
 5: 00000101    3: 00000011    5: 00000101
    --------      --------      --------
AND 00000100   OR 00000111  XOR 00000010
Resultado: 4   Resultado: 7  Resultado: 2
```

O resultado de cada operação fica no primeiro registrador. AND, OR e XOR comparam os bits correspondentes dos dois valores; não tratam os números inteiros como condições lógicas.

5. Evolução durante a execução

A tabela mostra os valores em decimal, depois de cada instrução. PC antes é o endereço buscado naquele ciclo; PC depois indica o endereço seguinte.

Ciclo	PC antes	Instrução	R0	R1	R2	R3	PC depois
1	0	LOAD R0, 12	12	0	0	0	1
2	1	LOAD R1, 5	12	5	0	0	2
3	2	ADD R0, R1	17	5	0	0	3
4	3	LOAD R2, 3	17	5	3	0	4
5	4	SUB R0, R2	14	5	3	0	5
6	5	AND R0, R1	4	5	3	0	6
7	6	OR R0, R2	7	5	3	0	7
8	7	XOR R0, R1	2	5	3	0	8
9	8	JMP 10	2	5	3	0	10
10	10	STORE R0, 15	2	5	3	0	11
11	11	STORE R1, 14	2	5	3	0	12
12	12	HALT	2	5	3	0	12

Resultado final

R0 = 2 | R1 = 5 | R2 = 3 | R3 = 0
MEM[15] = 2 | MEM[14] = 5 | PC = 12 | HALT = True

No ciclo 3, R0 recebe o resultado da soma. A subtração acontece no ciclo 5, e as operações lógicas nos ciclos 6 a 8. No ciclo 9, o PC salta de 8 para 10. STORE grava os valores na memória sem mudar os registradores. HALT encerra a execução e mantém o PC.

O exemplo do enunciado

LOAD R0, 5; LOAD R1, 8; ADD R0, R1; STORE R0, 15; HALT. Resultado verificado: **MEM[15] = 13**. Esse exemplo está disponível com --programa exemplo.

6. Como executar e demonstrar

Abra o terminal na pasta do projeto. A simulação e os testes usam somente a biblioteca padrão do Python.

```
py main.py --passo
py main.py --resumo
py main.py --programa exemplo --resumo
py main.py --programa 8bits --resumo
py testes.py
```

No Windows desta máquina, use py. Em outros ambientes, python ou python3. No modo --passo, Enter executa a próxima instrução.

O que foi implementado

O código-base já trazia os registradores, a memória, o PC e as instruções LOAD, JMP e HALT. A implementação completa STORE e as cinco operações da ULA, com sua integração à CPU. Também inclui os programas, os testes e a visualização dos estados antes e depois de cada instrução, com modo passo a passo.

Verificação

Os **36 testes automáticos passaram**. Em testes.py, as verificações incluem todas as instruções, as mudanças do PC, a parada, os programas completos, os limites de 8 bits, as entradas inválidas e o modo passo a passo.

Arquivos principais

Arquivo	Responsabilidade
cpu.py	Estado da CPU e ciclo de execução.
ula.py	Operações aritméticas e lógicas.
programas.py	Programa próprio e demonstrações.
main.py	Terminal, tabela e execução passo a passo.
testes.py	Verificação automática do funcionamento.

Fonte: enunciado Trabalho AOC - CPU Simulada.pdf, UFC Sobral, AOC 2026.2, professor Wendley S. Silva; código-base fornecido na disciplina. A tabela de evolução foi obtida executando a implementação do projeto.